Verdia: Testing Two Changes to HINT for Clinical Trial Outcome Prediction

Eric Munkhtur
Verdia · HackRU 2026

October 11, 2026
HackRU 2026 technical report, version 1.1

Abstract

Verdia is a HackRU project about predicting clinical trial outcomes and exploring the biology behind a trial. We evaluated the published HINT model on the Phase II TOP dataset, then fine-tuned it with two changes to how it combines eligibility sentences: learning which sentences deserve more weight, and using the trial's drug and disease to guide those weights. A matched HINT control, fine-tuned from the same starting checkpoint, scored 0.63202 on test ROC-AUC, compared with 0.63231 and 0.63098 for our two versions. These small differences do not show a reliable improvement. We checked the saved predictions and confirmed that the new layers worked, but had little effect on the final scores. We also built a website that maps clinical trial entities into PrimeKG and runs our fine-tuned model on supported Phase II drug trials. This report explains what we changed, the Colab and laptop hardware we used, the results, and the limits of the work.

1 What We Wanted to Find Out

A clinical trial tests a treatment in people. Predicting whether it will meet its goals is difficult because the result depends on the drug, the disease, the patients, and the study design. Our question was narrow: **could we improve HINT by changing how it combines the sentences in a trial's eligibility rules?**

Eligibility rules describe who can or cannot join a trial. HINT's released code turns each sentence into a list of numbers, then averages those lists. This gives every sentence the same weight within its group. We thought a learned weighting system might preserve useful details that a simple average overlooks. The experiments below test that idea; they do not establish that it works better.

2 How HINT Works

HINT is an existing neural network from Fu and colleagues [1, 2]. It brings together three inputs:

- **Drug:** a representation of the molecule's atoms and bonds.
- **Disease:** disease codes and their relationships in a medical hierarchy.
- **Eligibility rules:** text describing who can join the trial.

It combines these inputs and biomedical information in a small interaction graph, then produces a score between zero and one. Higher scores mean the model predicts a greater chance of success. We have not shown that a score such as 0.7 means a reliable 70% chance for a future trial.

BioBERT is the existing biomedical text model used to make the sentence representations [3]. We reused its saved outputs. We did not train BioBERT or build HINT from scratch as a new architecture. Another published approach, SPOT, learns from relationships and changes across historical trials [4]; we did not run SPOT in this project.

3 Data We Used

We used the Phase II files supplied by the HINT repository, at version `8dc0497`. TOP stands for *Trial Outcome Prediction*. The data already contains outcome labels. We kept its training, validation, and test groups as supplied.

Table 1: The three data groups used in the experiments.

Group	Trials	Success labels	Failure labels
Training: used to learn	4,005	1,914	2,091
Validation: used to choose a saved model	446	208	238
Test: used to report final scores	1,654	918	736

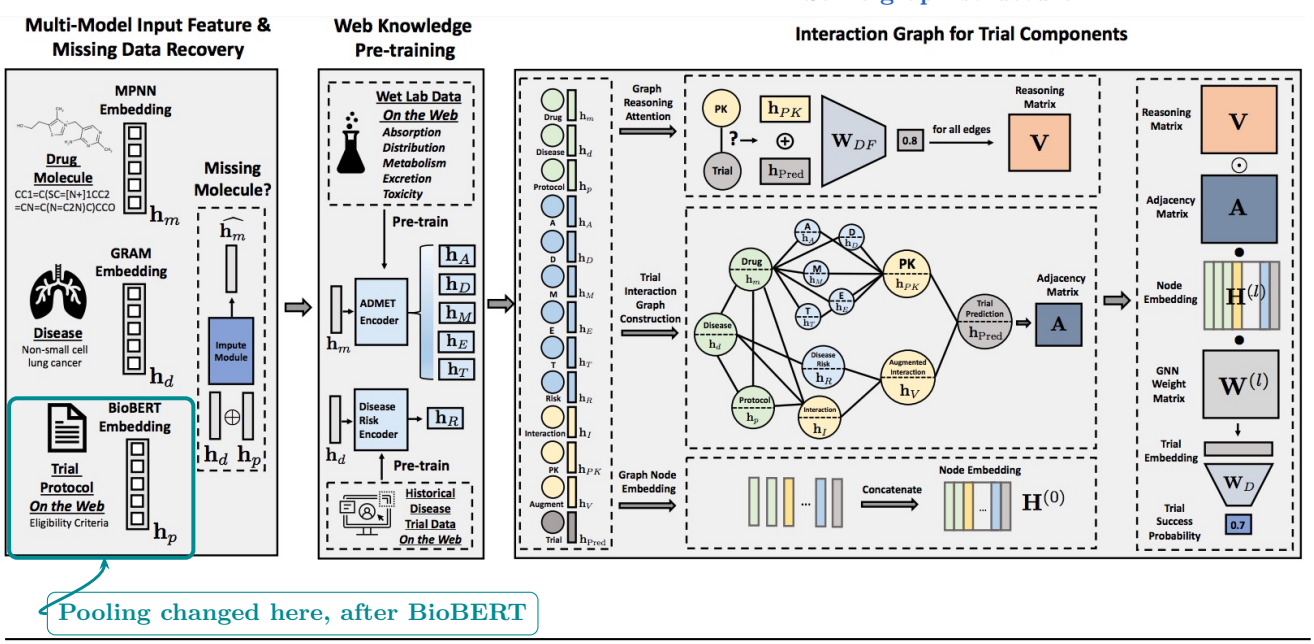
No trial identifier appears twice within a group or in more than one group. Related drugs and related trials could still appear across groups. The model reads drug structures, disease codes, and eligibility text. It does not receive trial status or the reason a trial stopped as input.

The reproduction work rebuilt the missing sentence-vector cache. A *cache* is a saved set of outputs so they do not have to be calculated again for each training run. The complete cache covered 513,863 sentences. The MSI used a smaller Phase II copy with 201,374 sentences, about 657 MiB, to fit its memory. The final Colab runs used a later regenerated Phase II cache with a different file hash from the MSI cache. We keep both identities in the evidence records; a different byte hash alone does not establish whether every numerical value differs. Within the MSI comparison, all three models used the same exact cache. We found no missing sentence entries in the Phase II data. The authors' original full cache was unavailable, so we cannot claim our rebuilt file is identical to it.

4 What We Changed

Figure 1 marks the part of HINT we changed. The original architecture image comes from the authors' GitHub repository. The colored outline and explanation boxes identify our additions; the original diagram itself is retained.

Where Verdia changed HINT



Original HINT	Attempt 1	Attempt 2
Give every eligibility sentence equal weight.	Learn which sentences should receive more weight.	Use the drug and disease to help choose sentence weights.

Figure 1: Our two attempts were separate experiments. Both changed sentence pooling—the step that combines sentence representations. BioBERT's saved sentence vectors stayed fixed. The rest of HINT kept the same architecture and was trained further. Source image: Fu et al., `HINT/hint.png` in the HINT repository [2]. Colored annotations added for Verdia.

4.1 Attempt 1: learn sentence weights

We replaced equal averaging with *attention*: a small learned scoring layer that gives some sentences more weight than others. There is one scoring rule for inclusion sentences and another for exclusion sentences. The rules are shared across trials. This added 1,536 trainable values to HINT.

4.2 Attempt 2: use the drug and disease as context

We kept the same idea, but also gave the weighting layer HINT's drug and disease representations. The goal was to let the same eligibility sentence receive a different weight in different trial contexts. This added 31,040 trainable values. It used the same saved sentence vectors and the same final 50-number protocol representation as HINT.

Both versions started with equal sentence weights, so they began with almost the same output as the original model. We changed the pooling code, not the graph's connections. A high attention weight means the model gave a sentence more weight; it does not prove that sentence caused a trial to succeed or fail.

5 Where We Ran the Work

Table 2: Hardware and its role in this project. GPU means graphics processor; VRAM is its memory.

Place	Hardware	What it did
Google Colab Pro, GPU session	NVIDIA Tesla T4, 16 GB VRAM, recorded in the run notes	Generated BioBERT sentence vectors. Earlier HINT GPU training attempts were interrupted and are not reported as completed results.
Google Colab, final CPU sessions	2 CPU cores; no active GPU in the saved environment	Completed the three original-HINT training runs reported below. Also used for checkpoint evaluation and result packaging.
MSI laptop	NVIDIA GTX 1050 Ti, 4 GB VRAM; about 8 GB system RAM	Trained the matched original-HINT control and both attention versions. The second attention run took about 28.7 minutes.
Azure virtual machine	Standard B2als v2; 2 CPU cores, 4 GB RAM	Hosted the Verdia website and ran live model predictions on CPU. It did not train the models in these tables.

Why some Colab work used CPU. The run notes record repeated session losses during earlier attempts. Completed results came from later CPU sessions, with model files saved during training. Those runs took about 26.5, 32.9, and 28.0 minutes each according to their runner logs. These times do not include the interrupted attempts or setup. The T4 is named in the earlier run notes; the final environment file describes the later CPU session.

Total time and compute. The final runtime log covers about 4 hours 52 minutes, from 10:23 PM on October 10 to about 3:15 AM on October 11, 2026 (New York time). This includes setup, interrupted attempts, data transfers, training, and downloading the evidence. The three completed Colab runners total about 87.4 minutes of work; some ran at the same time, so that is not a wall-clock duration. The log records roughly 2 hours 28 minutes with a T4 attached, including setup and waiting. Its estimate of 5–8 Colab compute units is an estimate, not a verified billing total.

Why the full task took longer. The runtime log records lost sessions, a duplicate training process, an embedding-format error caught by an equivalence check, and a failed cache save caught by its file hash. Large cache uploads also had to be split into smaller pieces. Interrupted runs are excluded from the result tables. These are limitations of this execution, not a general claim about how Colab behaves for every user.

Software. The final Colab environment records Python 3.13.16 and PyTorch 2.11.0+cpu. The MSI used Python 3.11.16 and PyTorch 2.4.1+cu121. These are separate environments, so we compare our attention versions against the control trained on the same MSI setup.

6 How We Kept the Comparison Fair

A *checkpoint* is a saved model. An *epoch* is one pass through the training data. A *seed* controls random choices such as the initial weights and training order.

For the MSI comparison, all three versions started from the same authors' Phase II checkpoint. This was *fine-tuning*: continuing to train an existing model on the trial-outcome task, with our new pooling layers added in the two experimental versions. Each used the same data, sentence vectors, seed, and five training epochs. We used batches of 32 trials and the Adam optimizer with a learning rate of 0.0003. All model weights could update during training, while the cached BioBERT vectors stayed fixed.

We chose the saved model with the lowest validation error, including the starting model as an option. All three selected the model after epoch four. The test set was then used to produce the reported scores. The three new Colab training runs were a separate reproduction task: they started with the released ADMET initialization, rather than the already-trained Phase II outcome checkpoint.

A recorded seed issue. The updated Colab report says run 0 used a script that set its seed before importing HINT. Those imports reset random generators. Runs 1 and 2 used the corrected order. We keep all three results, but describe them as three recorded runs, not a clean study where every seed was set by the same procedure.

A limit of the second attempt. We designed it after seeing the first attempt's test score. We fixed its settings before starting, but its result is still exploratory. It needs a new, untouched evaluation set before supporting a strong improvement claim.

7 What the Scores Mean

- **ROC-AUC:** measures how well the model ranks successful trials above failed trials. Higher is better; 0.5 is chance-level ranking and 1 is perfect ranking.
- **Average precision (AP):** measures how well the ranking brings successful trials toward the top. It depends partly on how common successful trials are in the data.
- **F1:** balances precision and recall after turning scores into yes/no predictions. We used 0.5 as the cutoff in every reported run.

We calculate AUC and AP from the model's original probability scores. The upstream code also reports a value called PR-AUC after replacing those scores with zeros and ones. We label that *thresholded AP* and keep it separate. On the authors' saved predictions, regular AP is 0.69065 and thresholded AP is 0.62654. That difference comes from the calculation, not from an improved model.

8 Results

8.1 Reproducing the original HINT

The authors released both a prediction file and a saved model. They did not give identical results in our setup. We report them separately from the models trained during this project.

Table 3: Original-HINT results on all 1,654 test trials.

Source	ROC-AUC	AP	F1
Authors' saved predictions, scored by us	0.64572	0.69065	0.61913
Authors' checkpoint with our rebuilt cache	0.62045	0.67273	0.63671
Colab training run 0	0.63025	0.67034	0.64891
Colab training run 1	0.63904	0.67592	0.64965
Colab training run 2	0.63912	0.68229	0.59499
Average of the three Colab run scores	0.63614	0.67619	0.63118

Additional check with no eligibility vectors. The reproduction workflow also evaluated the authors' checkpoint with the empty upstream cache, which makes HINT use zero vectors for the eligibility input. Test AUC was 0.62065 and AP was 0.67240, compared with 0.62045 and 0.67273 using the rebuilt cache. The ranking metrics barely changed in this checkpoint. This supports checking the importance of that input before changing its pooling, but does not prove that eligibility text is unimportant in every trained model. This check is separate from our MSI attention experiments.

The last row averages the scores of three models. It does not combine their predictions into a new model. We have not completed an ensemble experiment. We also cannot prove why the authors' saved model and saved predictions differ. Our rebuilt sentence cache and the code's model-saving behavior are possible contributors.

8.2 Testing our changes on the MSI

Table 4: Same starting checkpoint and training settings for all three MSI models.

Model	ROC-AUC	AP	F1
Fine-tuned HINT control	0.63202	0.68293	0.60584
Attempt 1: sentence attention	0.63231	0.68335	0.60559
Attempt 2: context attention	0.63098	0.68250	0.59603

Attempt 1 was almost tied with the control. Attempt 2 was slightly worse. We checked how much the AUC differences change when test trials are repeatedly resampled. The uncertainty ranges crossed zero for both attempts. This means the results do not give clear evidence that either change improves the original model. It also does not prove that attention could never help under different conditions.

9 Why the Changes Did Not Help Much

The new layers did work. Their weights changed during training. In the second version, changing the drug and disease input to the attention layer also changed the sentence weights. The issue was not simply an unused layer.

The final predictions barely moved. Compared with the MSI control, the average probability change was about 0.76 percentage points for attempt 1 and 1.04 points for attempt 2. The second version mostly lowered scores. At the 0.5 cutoff, it corrected five wrong classifications but made fifteen previously correct ones wrong.

Switching attention off had a small effect. On the validation set, we temporarily restored equal averaging while keeping each model's other learned weights. AUC changed from 0.71241 to 0.71124 for attempt 1, and from 0.71206 to 0.71122 for attempt 2. This suggests that learned pooling made only a small contribution in these trained models.

We changed weighting, not the available evidence. Both attempts reused the same fixed sentence vectors and the same HINT graph. We had assumed that averaging sentences was a major weakness. The results did not support that assumption strongly enough. Redundant information and the limited training data are possible explanations, not proven causes.

We found a shared text-parsing problem. When HINT cannot separate inclusion and exclusion headings, it puts the same complete text into both groups. That happened in 570 training, 73 validation, and 28 test trials. All three MSI models shared this behavior, so it does not by itself explain the difference between them.

10 The Verdia Website

Verdia also provides a graph explorer. A user enters an NCT number, the identifier for a ClinicalTrials.gov study. The application retrieves the trial, identifies its drug and disease, matches their names to PrimeKG, and displays connecting paths.

PrimeKG is an existing biomedical knowledge graph [5]. Our local copy has 129,375 nodes and 4,050,249 relations. The search follows up to two links outward from each side and shows up to 24 connecting paths. It uses relation types and a penalty for very highly connected nodes to choose routes.

The three-dimensional positions make the graph easier to explore. They are not the physical positions of molecules. The displayed paths can suggest relationships to inspect, but they do not prove a drug will work. The website now runs our sentence-attention model and the matched HINT control on CPU for supported Phase II drug trials. It resolves drug structures through PubChem, matches disease names to a dictionary built only from TOP training inputs, and encodes the current eligibility text with BioBERT. Missing or ambiguous inputs produce an unavailable message instead of a score. The page shows each model's score, the inputs used, and a downloadable run record. Saved historical scores and benchmark accuracy are labeled separately.

A live demonstration for NCT01132482 produced scores of 0.657 for our model and 0.664 for the control; the model worker took 2.64 seconds after input resolution. This is one observed request, not a latency benchmark. That trial is in TOP's test set, and its current registry text may differ from the historical inputs. The new live preprocessing has not been evaluated as a separate benchmark. PrimeKG paths are not inputs to either checkpoint. We have not measured name-matching accuracy or tested the website with clinicians.

11 Limits and Next Steps

Our attention comparison used one seed, one trial phase, and a short training budget. The second attempt was chosen after seeing earlier test results. The original embedding cache was unavailable, and some Colab setup records required corrections. These limits matter when interpreting small differences.

Future experiments could improve the eligibility parser, repeat the attention comparison with more seeds, or add information from earlier related trials. Any historical evidence must have been available before the trial's prediction date. None of those proposed improvements has a result in this report.

12 Conclusion

We evaluated and retrained HINT in our environment, fine-tuned two versions with changes to eligibility pooling, and built a website combining graph exploration with live model inference. Neither pooling change clearly improved prediction. The useful outcome is a documented experiment: the models ran, their outputs were saved, and the unsuccessful changes can be inspected. Training a predictor that uses the website's PrimeKG paths remains future work.

Code, Evidence, and Assistance

The project keeps model files, settings, training logs, per-trial predictions, and analysis outputs. An evidence manifest records file hashes so the exact inputs and outputs can be identified. Source code, this report as PDF and LaTeX, aggregate metrics, training configurations, and selected evidence records are published at `https://github.com/Eric-Munkhtur/verdia`. The original per-trial prediction files, checkpoint weights, and sentence caches are retained locally; their identities are recorded, but they are not included in this public release. A complete reproduction from the packaged repository has not been run, and the earlier Colab seed-0 script was not separately archived. The original HINT and PrimeKG work is credited in the references. The website is available at `https://verdia-hackru-20839887.westus2.cloudapp.azure.com/`.

AI assistants helped with coding, running experiments, analysis, and drafting. Eric Munkhtur is the named author. Reported scores come from saved experiment outputs. This technical report documents completed HackRU experiments; the system has not undergone peer review or clinical validation.

References

[1] T. Fu, K. Huang, C. Xiao, L. M. Glass, and J. Sun. HINT: Hierarchical interaction network for clinical-trial-outcome predictions. *Patterns*, 3(4):100445, 2022. doi:10.1016/j.patter.2022.100445.

[2] HINT authors. Clinical trial outcome prediction repository. `https://github.com/futianfan/clinical-trial-outcome-prediction`. Code and architecture image examined at commit `8dc0497`, October 2026.

[3] J. Lee et al. BioBERT: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 2020. doi:10.1093/bioinformatics/btz682.

[4] Z. Wang, C. Xiao, and J. Sun. SPOT: Sequential predictive modeling of clinical trial outcome with meta-learning. arXiv:2304.05352, 2023. `https://arxiv.org/abs/2304.05352`.

[5] P. Chandak, K. Huang, and M. Zitnik. Building a knowledge graph to enable precision medicine. *Scientific Data*, 10:67, 2023. doi:10.1038/s41597-023-01960-3.

A Details for Reproducing the Runs

This appendix keeps the exact settings available without putting equations in the main explanation.

- MSI: five epochs, seed 0, batch size 32, Adam, learning rate 0.0003, zero weight decay, binary cross-entropy training loss. Selection used average validation loss per trial.
- Colab: five epochs per completed run, batch size 32, Adam, learning rate 0.0003. Selection used the sum of average losses for each validation batch. Run 0's seed-setting issue is described in Section 6.
- Model sizes: original 3,172,995 parameters; attempt 1 adds 1,536; attempt 2 adds 31,040. Attempt 2 uses a 100-number drug/disease context and a 32-number internal attention representation. BioBERT vectors have 768 numbers per sentence.
- Uncertainty check: 1,000 paired resamples of saved test predictions, keeping success/failure counts fixed, random seed 20261011. The 95% AUC-difference ranges are [minus 0.001303, plus 0.001736] for attempt 1 and [minus 0.002506, plus 0.000452] for attempt 2. These ranges do not measure training-seed variation or account for related trials.
- Spread of the three recorded Colab run scores, using population standard deviation: AUC 0.00416, AP 0.00488, F1 0.02559. Because the seed procedure differed for run 0, these are descriptive summaries only.

Table 5: MSI validation error by epoch. Lower is better; epoch zero is before training.

Epoch	Original HINT	Attempt 1	Attempt 2
0	0.634962	0.634962	0.634962
1	0.628384	0.628246	0.628515
2	0.626424	0.626415	0.626335
3	0.625238	0.625643	0.629454
4	0.621753	0.621373	0.621629
5	0.623654	0.623856	0.623896

B File Identities

These identifiers link the MSI experiments to the exact source and saved inputs. They are file fingerprints, not model equations.

Upstream code version
`8dc0497f23fdb84e2905da7655924a91e6e79798`

Starting model SHA-256
`da2c94feca23c751925faceab118895b8ccb68c65a454daad01e1977d7af8e63`

MSI sentence cache SHA-256
`d4c8f2301ef0eb3213eac9de4c8a13c32a7e5d43efa8da292ab9d230d9e54a90`

Final Colab sentence cache SHA-256
`61e5933cb7ff91d59d4f29f6dce17a6bb818ebe4bb6968517f45b1e2fee3602b`

Second-attempt settings lock SHA-256
`46bceb2616c0953d17205da3e80a4c2ac4efaabafbfd249a29f5f0be59205217`

C Runtime Record

The figures in Section 5 come from the final `RUNTIME_ACCOUNTING.md`, compiled from session history and runner logs. This appendix summarizes the seven sessions. Session lengths include preparation and transfers; they are not all training time. Several sessions overlapped.

Session	Duration	Work and outcome
Initial CPU	About 4 hours	Code and data checks, scoring released predictions, embedding checks.
First T4	About 63 min	Full sentence cache generation and early training attempts; training interrupted.
Second T4	About 65 min	Cache regeneration and checkpoint evaluation; training interrupted.
Third T4	About 20 min	Phase II cache regeneration and transfer; stopped after completion.
First training CPU	About 62 min	Completed run 0; a later run 2 attempt was interrupted.
Second training CPU	About 63 min	Completed run 1.
Third training CPU	About 58 min	Completed run 2, saved the environment, and packaged results.

Exact result tables use the saved metric files. Rounded summaries in the runtime log do not override those files. Completed runner times were 1,592.2, 1,973.5, and 1,677.2 seconds. The runtime estimate also includes failed attempts; their partial outputs are not treated as additional completed experiments.